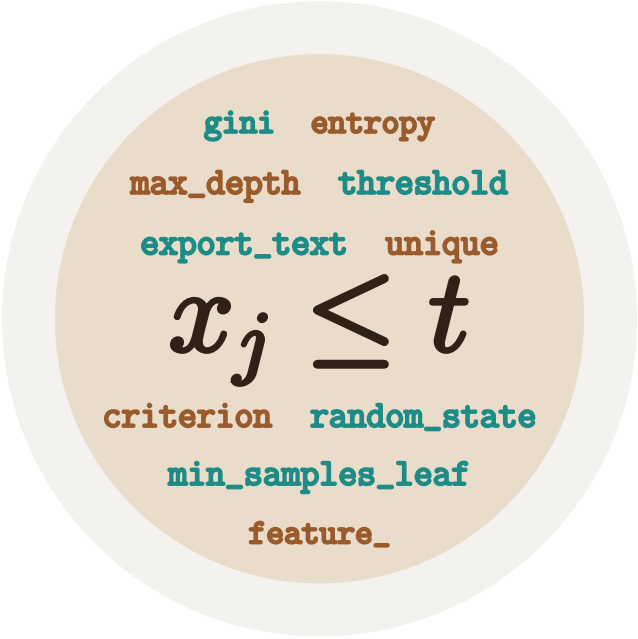
Решающие деревья

Шпаргалка



Формулы и вызовы из урока «Решающие деревья». Строка с формулой — что это, формула, ниже пример. Строка с кодом — что ищут, что делает, ниже пример и после стрелки его значение, снятое запуском. Примеры кода — на этих массивах:

```
import numpy as np
X = np.array([[1, 2], [2, 0], [4, 4], [5, 2]])
y = np.array([0, 0, 1, 1])
r = np.array([1.0, 2.0, 3.0, 5.0])
q = np.array([2, 2])
```

Примесь и вопрос

Джини — $G = 1 - \sum_c p_c^2$; при K равных классах $1 - \frac{1}{K}$; вероятность, что два объекта узла разных классов
▷ доли $\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$: $G = \frac{5}{8}$

Энтропия — $H = -\sum_c p_c \log_2 p_c$, слагаемое при $p_c = 0$ — ноль; при K равных классах $\log_2 K$
▷ доли $\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$: $H = \frac{3}{2}$

Прирост — $\Delta = I - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R)$, лучший вопрос — наибольший Δ
▷ классы $(0, 0, 1, 1)$, первый объект отделён: $\frac{1}{2} - \frac{3}{4} \cdot \frac{4}{9} = \frac{1}{6}$

Пороги — середины между соседними различными значениями; m значений — $m - 1$ порогов
▷ значения $1, 3, 3, 8$: пороги 2 и 5,5

`np.bincount(y) / len(y)` — доли классов в узле

```
np.bincount(y) / len(y) → [0.5 0.5]
```

`1 - (p ** 2).sum()` — Джини по долям

```
1 - ((np.bincount(y) / len(y)) ** 2).sum() → 0.5
```

`np.unique` — различные значения признака

```
v = np.unique([1, 3, 3, 8])
```

`(v[1:] + v[:-1]) / 2` — середины — пороги

```
(v[1:] + v[:-1]) / 2 → [2.  5.5]
```

Построение

Жадно — лучший вопрос в узле, потом в каждой части отдельно, как в целой выборке
▷ вопрос выбирают по приросту сейчас, не заглядывая ниже

Лист — один класс, никакой вопрос не делит или сработал останов; ответ — самый частый класс
▷ останов — глубина, мало объектов в узле, малый прирост

Регрессия — примесь — дисперсия y в узле, ответ листа — среднее
▷ $y = (1, 3, 5)$: дисперсия $\frac{8}{3}$, лист отвечает 3

Масштаб и глубина — масштаб не важен: $x_j \le t$ после монотонного преобразования — тот же вопрос с другим t; глубокое дерево учит шум
▷ масса в килограммах — порог 4,3 вместо 4300

Вызовы sklearn

Порядок — как у всех моделей: создать, `fit(X, y)`, `predict(X)`. Модуль — `sklearn.tree`.

`DecisionTreeClassifier` — `max_depth`, `criterion="entropy"`, `random_state`

```
tr = DecisionTreeClassifier(max_depth=1).fit(X, y)
```

`tree_.feature` — признак вопроса в узле; корень — `[0]`

```
tr.tree_.feature[0] → 0
```

`tree_.threshold` — порог вопроса в узле

```
tr.tree_.threshold[0] → 3.0
```

`export_text` — дерево текстом, `feature_names=`

```
txt = export_text(tr)
```

`min_samples_leaf` — наименьший лист — ещё один останов

```
DecisionTreeClassifier(min_samples_leaf=2)
```

`DecisionTreeRegressor` — примесь MSE, лист отвечает средним

```
dr = DecisionTreeRegressor(max_depth=1).fit(X, r)
```

`dr.predict` — ответ листа

```
dr.predict([q]) → [2.]
```

Где sklearn расходится с точными правилами

Ничьи — из вопросов с равным приростом — первый в случайном порядке признаков, его задаёт `random_state`
▷ два одинаковых столбца: в корне то один, то другой

Пороги — признаки хранятся в `float32`, с семью значащими цифрами; порог — середина уже округлённых чисел
▷ между 0,2 и 0,30000001 — 0,2500000075